

PURCHASING AGENT

Approach, Architecture & Evaluation

AI Purchasing Agent Assignment

A buyer-support system for quick-commerce purchasing decisions that can inspect operational data, make a constrained decision, execute an authorized purchase action, validate the result, and escalate uncertain or high-risk cases to a human buyer.

Prepared by	Chirag Gill
Repository	github.com/mick-coder12/ai-buyer-agent
Live demo	ai-purchasing-agent-vsun.onrender.com
Stack	Python 3.11 • FastAPI • SQLite • Vanilla JS • pytest • Render

Core design principle

The system does not blindly accept an upstream purchasing recommendation. It independently checks inventory, demand, supplier terms, budget and storage before acting.

This document is based on the implementation currently present in the public repository and the seeded scenarios used by the application.

SYSTEM OVERVIEW

1. Executive Summary

The Purchasing Agent is designed as a decision-and-execution system rather than a conversational chatbot. For a given purchasing situation, the agent gathers the relevant operational facts, applies deterministic purchasing policies, performs an action when it is within its autonomy boundary, and then validates that the action still satisfies the constraint it relied on.

The implementation intentionally keeps the accept/modify/reject/escalate decision logic deterministic and auditable. A live language model is not required at decision time. This makes the same input reproducible, allows the policy layer to be tested like ordinary application code, and exposes the exact factors behind each decision.

4

assignment scenarios

5

seeded situations

7

suppliers

\$5K

auto-action cost threshold

10%

alternate supplier premium ceiling

3 days

sustained demand signal

What makes the implementation useful

- **Independent verification:** recommendations are treated as inputs to inspect, not as ground truth.
- **Constrained execution:** purchase orders are created or modified only through the agent's tool boundary.
- **Closed-loop validation:** the agent re-reads critical constraints after acting and can correct a safe discrepancy.
- **Human-in-the-loop:** uncertain, expensive, or constraint-conflicting cases are routed to a buyer with the reasoning trace.
- **Reproducibility:** SQLite is re-seeded from backend/seed_data.json at startup.

WHY THE SYSTEM IS STRUCTURED THIS WAY

2. Problem & Approach

The purchasing problem

A purchasing system may propose an order using a forecast or predefined rule, but the proposal can become incorrect when inventory, open orders, supplier availability, demand, budget, or warehouse capacity differ from the assumptions behind it. The agent therefore answers a more fundamental question: **how much should be bought, from whom, and can the system safely execute that decision right now?**

The five-step decision loop

Step	Purpose	Implementation
1. Perceive	Gather current operational facts.	Tools read product, demand, suppliers, open POs, budget and storage.
2. Reason	Determine the appropriate action.	Policy engine runs <code>evaluate_purchase</code> or <code>evaluate_shortfall</code> .
3. Act	Execute only within the autonomy boundary.	Create/update a purchase order, or create an approval request.
4. Validate	Check whether the action still fits.	Re-read the relevant constraint and correct or fail closed.
5. Escalate	Hand uncertain cases to a buyer.	Approval queue exposes proposal + reasoning for approve/reject.

Design choice: Scenarios 1, 3 and 4 share `evaluate_purchase` because they are different constraints on the same purchasing decision. Supplier shortfall is handled separately by `evaluate_shortfall` but still uses the same `act` → `validate` → `escalate` pattern.

SYSTEM DESIGN

3. Architecture

The implementation uses a small layered architecture so the policy logic stays independent from storage, HTTP and UI concerns.

Browser UI Situations • reasoning trace • approval queue • purchase orders	FastAPI application HTTP API • request handling • orchestration entry points
Agent orchestrator Perceive → Reason → Act → Validate	Human buyer Approve / reject escalated proposals
Policy engine evaluate_purchase evaluate_shortfall	Tools layer Inventory • demand • suppliers • budget • storage • PO operations • audit log
SQLite Products • suppliers • situations • purchase orders • approvals • audit log	Seed data backend/seed_data.json keeps demo state reproducible

Technology choices

Layer	Choice
Language	Python 3.11
API	FastAPI + Uvicorn
Data	SQLite using stdlib sqlite3; no ORM
Frontend	Vanilla HTML / CSS / JavaScript; single page; no build step
Testing	pytest + scenario evaluation script
Hosting	Render free web service

The repository exposes the backend, frontend, deployment configuration and the approach document directly. The live demo is deployed from the same repository.

HOW THE AGENT BEHAVES

4. Scenario Walkthroughs

Scenario	Starting condition	Agent behaviour / outcome
SIT-1001 Recommendation review	Recommendation: 800 units. Inventory + open order imply ~338 units are needed. Storage has only 300 units available.	Modify to 300 and execute. During validation, storage falls by 40 units; the PO is safely corrected to 260 because the supplier MOQ is 150.
SIT-1002 Correct recommendation	Recommendation: 150 units. Agent calculation is ~144 and constraints do not block the order.	Accept the recommendation and place the order. Demonstrates that the agent does not change recommendations without a material reason.
SIT-2001 Supplier shortfall	Existing order is 500 units, but supplier confirms only 250. A 250-unit gap remains.	Check whether the gap is safe to ignore; source the shortfall from an alternate supplier when the price premium and cost stay within policy.
SIT-3001 Demand changed	Actual demand has risen sharply versus the forecast and the change is sustained.	Re-base demand using actuals, calculate a new purchase recommendation, and send it to a human buyer rather than auto-executing the forecast override.
SIT-4001 Purchasing constraint	Suggested order exceeds what the remaining budget can support, and the supplier MOQ is above the affordable quantity.	Do not place an invalid PO. Escalate with the constraint analysis so a buyer can request budget, delay, or choose another supplier.

Key observation: the system is not designed to maximize autonomous actions. It is designed to make autonomous actions only when the order is within explicit policy boundaries.

THE MOST IMPORTANT CONTROL

5. Feedback Loop & Self-Correction

The strongest demonstration of the design is SIT-1001, where the agent's first action becomes invalid before confirmation. Instead of assuming that its earlier read is still true, it validates the shared constraint again.

Stage	What happens
1. Initial reasoning	800-unit recommendation is reviewed. Inventory and lead time imply ~338 units are needed, but storage allows only 300.
2. Action	Agent creates a 300-unit purchase order because the order is within its autonomy boundary.
3. Re-check	Validation reads storage again. The seeded scenario makes 40 units unavailable, leaving 260.
4. Correction	Because 260 is still $\geq$ the supplier MOQ of 150, the agent updates its own PO from 300 to 260.
5. Fail-safe	If correction would fall below MOQ, the agent escalates instead of leaving a broken order or guessing.

Why this matters

- The system treats external state as changeable, even within one workflow.
- Validation is part of the business operation, not merely a logging step.
- The correction is bounded by the same supplier constraints used during the original decision.
- If the safe correction cannot be made, the system fails closed and asks a human to decide.

OPERATIONAL BOUNDARIES

6. Tools, Policy & Human-in-the-Loop

Tool boundary

Tool / operation	Purpose
get_product	Read inventory and demand information for a SKU.
get_suppliers	Read supplier price, lead time, MOQ, reliability and availability.
get_open_purchase_orders	Account for inventory already on order.
read_storage_remaining	Read remaining capacity at the purchasing node.
create_purchase_order / update_purchase_order	Only mechanisms used by the agent to change purchasing state.
create_approval	Create a buyer approval request when autonomous action is not appropriate.
log	Record perceive/reason/act/validate events for the UI trace and audit trail.

Autonomy boundary

The current policy allows autonomous execution for accept/modify decisions when the resulting order is below **\$5,000**, demand is considered stable, and a compliant quantity exists at or above the supplier minimum. Otherwise the proposal is routed to a buyer.

Condition	Current handling
Order cost $\geq$ \$5,000	Human approval
Demand spike / forecast override	Human review
No quantity satisfies MOQ + budget/storage	Escalate
Alternate supplier premium > 10%	Propose rather than auto-place
Safe low-cost order with compliant quantity	Auto-execute

Important implementation note: these thresholds are policy choices in backend/app/policy.py, not claims that the values are universally correct for real purchasing operations.

EVIDENCE THAT THE BEHAVIOUR IS REPRODUCIBLE

7. Testing & Evaluation

The repository contains both automated tests and a standalone evaluation script. The seeded scenarios are run through the real agent rather than a simplified mock, and the self-correction quantity for SIT-1001 is explicitly asserted.

Situation	Expected decision	Status	Action	Validated	Escalated
SIT-1001	modify	resolved_with_adjustment	Yes	Yes	No
SIT-1002	accept	resolved	Yes	Yes	No
SIT-2001	sourced_alternate	resolved	Yes	Yes	No
SIT-3001	recommend_purchase	pending_approval	No	Yes	Yes
SIT-4001	escalate	escalated	No	Yes	Yes

Test coverage

- `backend/tests/test_scenarios.py` runs the seeded situations through the real agent and checks decision/status/action/validation behaviour.
- SIT-1001 additionally checks the exact post-validation quantity of **260** units.
- The policy layer includes synthetic coverage for a shortfall with no alternate supplier and a demand spike that has not lasted long enough to trust.
- `backend/tests/evaluate.py` prints a compact evaluation scorecard across the seeded scenarios.

Run commands

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

```
python -m pytest tests/ -v
python tests/evaluate.py
```

IMPLEMENTATION MAP

8. Data Model & Repository Structure

Seeded data

backend/seed_data.json contains the complete demo state: 5 products, 7 suppliers, 3 open purchase orders, and 5 purchasing situations. The database is rebuilt from this seed on startup so the scenarios can be reproduced consistently.

Area	Role in the system
Products	Current stock and demand-related information.
Suppliers	Price, lead time, MOQ, reliability and available quantity.
Purchase orders	Existing and agent-created purchasing commitments.
Situations	Seeded events that trigger the agent loop.
Approvals	Human decisions for escalated proposals.
Audit log	Trace of perceive/reason/act/validate operations.

Repository structure

Path	Purpose
backend/app/policy.py	Decision rules and thresholds.
backend/app/agent.py	Perceive → reason → act → validate orchestration.
backend/app/tools.py	Inventory, supplier, storage, PO and approval operations.
backend/app/main.py	FastAPI application and API endpoints.
backend/seed_data.json	Reproducible products, suppliers, POs and situations.
backend/tests/test_scenarios.py	Automated scenario tests.
backend/tests/evaluate.py	Readable evaluation scorecard.
frontend/	Single-page browser interface.
render.yaml	Render deployment configuration.

LOCAL + HOSTED WORKFLOW

9. Running & Demonstrating the System

Local setup

```
git clone https://github.com/mick-coder12/ai-buyer-agent
cd ai-buyer-agent/backend
```

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

```
uvicorn app.main:app --reload
```

```
# Open in browser:
http://localhost:8000
```

Demo flow

- Open the application and select one of the seeded purchasing situations.
- Run the agent and inspect the reasoning trace generated from the actual tool/policy flow.
- Review the decision and any purchase order created or modified.
- For escalated cases, use the approval queue to approve or reject the proposal.
- For the feedback-loop case, inspect how validation detects storage drift and corrects the PO.

Hosted demo

ai-purchasing-agent-vsun.onrender.com

The repository README notes that the demo uses Render's free tier; after inactivity, the first request may take time to wake the service.

Repository

<https://github.com/mick-coder12/ai-buyer-agent>

WHAT WOULD CHANGE FOR PRODUCTION

10. Limitations & Future Extensions

Current limitations

- **Single node / single buyer:** the prototype does not implement multi-tenant access control or multi-team workflows.
- **Simulated external systems:** supplier confirmations, budget and storage are represented by seeded data rather than live integrations.
- **Seeded storage drift:** the feedback-loop race is deliberately reproducible rather than produced by real concurrent processes.
- **No persistence across restarts:** re-seeding is intentional so every demo starts from a known state.
- **No live LLM in the decision path:** the current policy engine is deterministic. A model could be added behind the existing tool boundary for explanation or structured extraction.

Natural next steps

- Connect live inventory, forecasting, supplier and budgeting services behind the existing tools interface.
- Replace seeded storage drift with real transactional/concurrency-safe capacity checks.
- Add authentication, authorization and buyer-specific approval policies.
- Support multiple warehouses/nodes and location-aware supplier selection.
- Add an optional LLM layer for parsing supplier emails into structured shortfall events and generating buyer-friendly explanations.
- Add production observability, persistent audit storage, retries/idempotency and stronger transactional guarantees around PO updates.

Closing summary

The central idea is a closed-loop purchasing agent: inspect → reason → act → validate → correct or escalate. The implementation keeps the decision boundary explicit and testable while leaving clear seams for real operational integrations.

Prepared by Chirag Gill

[GitHub](#) • [Live Demo](#)